



is OXR-SDK AI ready?

OXR Research · PromptScene

>> Table of contents

1. 추진 배경
2. 추진 내용
3. 검증 결과
4. 결론
5. 향후 계획

>> 이 발표의 주장

프레임워크가 **AI-ready** 가 됐다는 이야기입니다. **OXR-SDK가 그 사례**입니다.

오늘 주인공은 XRCollabDemo가 아닙니다. XRCollabDemo는 OXR-SDK가 AI-ready라는 사실을 보여주는 **증거**입니다.

SDK의 문서와 패키지만으로, 사람의 암묵지 없이, AI 에이전트가 검증 가능하게 작동하는 산출물을 만들 수 있는 상태.

오늘 증명하는 것은 "**AI가 문서만 보고 작동하는 룸을 자율로 빌드·검증했다**" 입니다. "문장 하나로 임의의 룸을 합성한다"는 다음 단계(Phase 5)이며, 이 둘은 다른 이야기입니다.

01

추진 배경

DX에서 AX로 — 프레임워크의 새 평가축

개발의 병목이 옮겨가고 있다

과거의 관건은 "사람이 이걸 짤 수 있는가" 였습니다.

이제는 "AI가 이 프레임워크를 쓸 수 있는가" 입니다.

- 코딩·개발이 AI 에이전트 기반으로 이동 → **DX(개발자 경험)에서 AX(에이전트 경험)로의 전환**
- AI에게 읽히지 않는(legible하지 않은) 프레임워크는 마찰이 되고, 결국 **도태**됩니다.

그래서 프레임워크의 **AI-ready화**가 필요합니다. 그런데 OXR-SDK도 처음에는 AI-ready가 아니었습니다 — 저장소를 클론한 그대로는 **컴파일조차 되지 않았**습니다.

com.oxr.sdk 의 .asmdef.meta · .cs.meta 누락으로 DLL 미생성 · XumLobby 0.4.4가 사라진 구버전 OxrSdk API 호출 · 임포트 샘플의 타입 정의 결손 — 이 **세 함정**을 사람이 손으로 메워야 비로소 0에러 컴파일.

"그냥 클론해서 쓰면 되지 않나?"

정적 샘플 클론은 결과물을 통째로 **동결**해서 줍니다. 빠른 출발점을 주지만, 작동에 필요한 지식은 그 안에 **숨어** 있습니다. 한 발만 벗어나면 다시 내부 지식이 필요해집니다.

이 차이를 가장 분명히 보여준 것이 '**스포너 클론**' 발견입니다.

- 기존에 "되던" 룸이 작동한 이유 = 플레이어 스폰서를 원본 `Room.unity` 에서 **통째로 복제**해 왔기 때문 → 순수 문서만으로는 재현 불가
- 스크립트로 직접 만든 스폰서 → 유효한 scene id를 못 받아 입장 시 `"Failed to confirm the access"` 로 **거부**
- 프리팹으로 인스턴스화 → **통과**

이 숨어 있던 규칙을 문서와 계약으로 끄집어내 노출시키는 것 — 그것이 AI-ready로 가는 작업이었습니다.

02

추진 내용

문서 → 절차서 → AI·MCP 자율 빌드·검증

빈틈을 메운 절차서 한 편

OXR-SDK 패키지에 흩어져 있던 문서를 읽어, **작동하는 룸을 처음부터 조립하는 절차서 한 편**으로 정리했습니다.

- 기존 출처(패키지 README, 공식 gitbook)는 "룸 레시피"는 주지만 **스포너 구성과 작동 불변식이 빠져** 그대로는 돌지 않았습니다.
- 그 빈틈을 채워 넣었습니다 — **스포너는 프리팹이어야 한다**는 점 + **C1~C4 작동 불변식**.

이 문서 **하나만** 읽으면 입장·아바타 스폰·이동이 되는 룸을 만들 수 있습니다.

>> AI 에이전트와 MCP만으로 자율 빌드

사람이 클릭으로 만든 것을 흉내 낸 게 아니라, AI가 **MCP(Model Context Protocol)** 로 Unity 에디터를 직접 조작했습니다.

씬 생성 → 네트워크 프리팹 배치 → 스폰너 프리팹 인스턴스화 → 서버 실행파일 빌드 → 서버 기동·로그 판독 → 작동 검증 — **전 과정을 AI가 자율 수행.**

항목	값
AI 에이전트	Claude Code (claude-sonnet-4-6)
Unity 연동	MCP → Unity 에디터, 포트 27826
대상 프로젝트	XRCollabDemo, Unity 6000.3.11f1

증명한 것은 "문서만 보고 작동 룸을 자율로 빌드·검증". "자연어 한 문장 → 임의 룸 합성"은 이 위에 쌓을 다음 단계입니다.

03

검증 결과

그럴듯한 룸이 아니라, 증명 가능하게 작동하는 룸

룸이 서버에 정상 등록됐다

문서 하나만 보고 조립한 **BasicRoom**을 빌드·실행·검증한 결과입니다.

- 서버 로그에 `Online Scene: BasicRoom`, 클라이언트 `Client 0 is successfully validated`
- 마스터 서버 `192.168.50.49:5000` listening · `Spawner successfully created`
- 룸 서버 `:7777` 에서 `Room registered successfully. Room ID:`

등록된 방 — `Room-1A5C-ED20-3935` · **ID 0 · 최대 10명 · Public**

입장과 동시에 화면이 룸으로 전환됐습니다. 로비 씬(`Client`)이 언로드되고 BasicRoom이 로드되며 로비 UI(`c-MasterCanvas`)가 사라졌습니다 — **별도 처리 없이 자동 전환**.

아바타가 룸에 스폰됐다

NetworkObjects = 5

- 소유 아바타 `Desktop(Clone)` · 골격 `mixamorig:Hips` · 플랫폼 루트가 모두 `scene=BasicRoom` 에 위치
- 아바타 카메라가 활성화되어 게임뷰에 룸이 보였고, 로비가 화면을 가리지 않았습니다

별도 처리 없이 입장 → 씬 전환 → 아바타 스폰이 한 번에 — **문서 하나만 보고 조립한 룸**에서 작동했습니다.

즉 그럴듯해 보이는 룸이 아니라, C1~C4 기준으로 **증명 가능하게 작동하는 룸**입니다.

>> "검증 가능"의 실체 — 작동 불변식 C1~C4

판정에는 미리 정의된 기준이 있었습니다.

- **C1** 프리팹 컬렉션이 서버·클라이언트에서 일치
- **C2** 플레이어 스폰이 지정된 스폰너 프리팹으로 이뤄짐
- **C3** 룸 진입/이탈 시 씬이 정확히 전환
- **C4** 서버는 Master+Room 실행파일 · 에디터는 Client+Room 동시 로드 토폴로지 유지

검증 기준 자체가 **구조화**되어 있어, AI는 결과가 맞는지 틀린지를 **스스로 판단**할 수 있습니다.

패키지 README·공식 문서만으로는 재현되지 않았습니다. AI가 문서의 빈틈을 발굴·보완한 덕분에 처음 보는 사람도 따라 할 수 있는 절차가 완성됐습니다.

04

결론

OXR-SDK는 AI-Ready 되었다

AI-Ready 충족 — 근거 네 가지

정의 — 문서와 패키지만으로, 사람의 암묵지 없이, 검증 가능하게 작동하는 산출물을 AI가 만들 수 있는 상태 — 를 충족합니다.

1. 작동 규칙이 **명시적으로 문서화** (C1~C4, 스폰서=프리팍 규칙)
2. 확장 면이 **표준 계약**으로 정의 (`IToggleableContent` , 코어가 개별 기능을 모르는 플러그인 구조 — UE5 Modular Game Features와 같은 원리)
3. 검증 기준이 **구조화**
4. 사람의 암묵지 없이 AI가 **처음부터 작동 룸을 재현·검증**

여기서 XRCollabDemo의 위상이 바뀝니다 — 정적 다운로드 샘플에서, **재생성·검증 가능한 best sample** 레퍼런스로. 재생성된다는 사실 자체가 OXR-SDK가 AI-ready라는 증거입니다.

05

향후 계획

자율 빌드 위에 — 자연어로 기능을 구사하는 단계로

>> 콘텐츠를 붙이기 위한 토대 — 이미 증명됨

이번에 검증한 계약 구조는 사실 **나중에 콘텐츠를 붙이기 위해** 만든 것입니다 — 자연어로 룸·기능을 생성하는 다음 단계의 토대.

- 확장 면을 표준 계약 `IToggleableContent` 로 정의 — 코어가 개별 기능의 존재를 모르는 플러그인 구조 (UE5 Modular Game Features와 같은 원리)
- 측정 기능 **Ruler**가 그 첫 파일럿 — 계약 위에 재구현해 룸 코어에 자기등록, 거리 라벨 **4.00m** 정상 렌더
- 원본 프로젝트(DeepChairProject) 앱 레이어 의존 **0** — 계약 위 클린 재구현

이 토대(Phase 1:2 완료) 위에 자연어로 기능을 엮는 것이 Phase 3~5입니다.

다음 단계 — 자연어 기능 · 제너레이티브 UI

검증된 자율 빌드를 토대로, **자연어로 기능을 구사해 붙이는 단계**로 확장합니다.

- **DeepChairProject**처럼 기능을 자연어로 설명해 만들어 보기 (다음 목표)
- 기능을 자유자재로 붙이기 위한 **제너레이티브 UI** 구조 — 레지스트리에 등록된 콘텐츠를 아이콘 그리드로 그리고 스마트폰처럼 토글하는 **런치패드**가 출발점

로드맵

Phase	내용	상태
0	룸 베이스 (네트워크 + 아바타 + UI 전환, C1~C4)	완료
1	계약 규격 (IToggleableContent , 씬 계층 표준)	완료
2	Ruler 파일럿 (계약 위 클린 재구현·검증)	완료
2.5	씬 계층 표준화 (SYSTEMS / ENVIRONMENT / UI / FEATURES / _DYNAMIC)	완료
3	런치패드 UI — 레지스트리 → 아이콘 그리드 → 토크	예정
4	스킬화 — /RoomContent<feature> + LLM 신규 기능 생성 템플릿	예정
5	합성 + 하네스 — /promptscene (자연어 → 룸 조립) + C1~C4 자동 검증	예정

Phase 3~5가 닫히면 자연어 한 줄로 작동하는 협업 룸과 자동 검증 리포트를 얻습니다. 오늘은 그 **기반이 증명됐다**는 보고입니다.